

# SKILLRACE: Reasoning-Augmented Concolic Execution for Coding-Agent Skills

ANONYMOUS AUTHOR(S)

A coding-agent *skill* is reusable procedural guidance—a natural-language SKILL.md plus optional scripts—that an agent consults to carry out a task such as fixing a failing test or scaffolding a service. Skills are now a first-class, sharable artifact, yet they are trusted after a handful of successful runs and tested almost not at all: unlike code, a skill has no compiler and no test suite, and the guidance that works on the situations its author happened to try may mislead an agent on the situations its author did not. We present SKILLRACE, which tests a skill the way concolic execution tests code. SKILLRACE runs the skill, lifts each run into a sequence of *episodes*—coherent units of work, each with a purpose and an outcome read from tool output rather than from the agent’s own narration—and merges episodes into a growing *behavior tree*. It reads the agent’s reasoning at each branch as the *condition* that sent the run one way rather than another, then mutates those conditions to synthesize new (prompt, environment) inputs intended to drive the skill toward branches it has not yet taken. Candidate starting states are validated against the mutated environmental condition *before* any agent run; whether execution reaches the intended branch is recorded afterward, not required for a discovered defect to count. Correctness is judged by *properties*: universal process invariants checked mechanically, and per-skill specifications compiled into executable checks *before* the run they judge, so that no model ever grades the run it also produced. We evaluate SKILLRACE against independent random generation and a VeriGrey-inspired rung that ports the feedback mechanism of a recent agent fuzzer to this setting, on a suite of 30 skills mined from public repositories. We repeat the complete comparison in two model-frozen tracks and report them separately. Holding the runner, environments, budget, and oracle identical across methods within each track, SKILLRACE exposes TBA more distinct defects per agent-run budget than the VeriGrey-inspired rung and TBA more than Random, and reaches its first violation in fewer runs. Feeding SKILLRACE’s findings back to revise an LLM-generated skill raises its pass rate on independently-authored held-out tests from TBA to TBA.

CCS Concepts: • **Software and its engineering** → **Software testing and debugging**.

Additional Key Words and Phrases: coding agents, agent skills, concolic execution, software testing, coverage-guided testing

## ACM Reference Format:

Anonymous Author(s). 2026. SKILLRACE: Reasoning-Augmented Concolic Execution for Coding-Agent Skills. 1, 1 (July 2026), 13 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

## 1 INTRODUCTION

Coding agents increasingly rely on *skills*: reusable folders of natural-language guidance, and sometimes helper scripts, that an agent loads on demand to perform a recurring task [1, 2]. A skill for fixing a failing test tells the agent to locate the failure, understand the expected behavior, edit the implementation, and re-run the suite; a skill for scaffolding a service prescribes a project layout and a build step. Skills are authored once, published to shared directories, and reused across many tasks and many agents [1]. In this they resemble a library—code that many downstream users

---

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](mailto:permissions@acm.org).

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM XXXX-XXXX/2026/7-ART

<https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

depend on—except that a skill is prose, its “interface” is whatever situations its author imagined, and it ships with neither a type checker nor a test suite.

This creates a testing gap that is easy to state and hard to close. A skill is trusted once it has worked on a handful of tasks its author tried by hand. But the tasks an author tries are the tasks an author thinks of, and a skill’s guidance is only as good as its coverage of the situations an agent will actually meet. Guidance that is silent about a broken build, ambiguous about which of two files to edit, or that assumes a test harness runs cleanly will send an agent confidently down the wrong path exactly when the environment departs from the author’s mental model—and because the agent narrates its own progress, the failure is often invisible in the transcript the author reads. What is missing is a way to ask, systematically, *on which situations does this skill’s guidance break down?*

The analogous question for code has a mature answer. Symbolic and concolic execution run a program on abstract inputs, read the branch conditions off the source, and negate them to synthesize inputs that drive execution down not-yet-taken paths [4, 9, 16, 20]. Coverage of the path space is both the goal and the search signal: the path tree built so far reveals the next frontier to push toward. This is precisely the discipline a skill lacks—and it does not transfer directly, because a skill is not code. There is no source to read a branch condition from; the “program” is an agent whose control flow is mediated by natural-language reasoning over the situations it observes.

Our key observation is that the agent’s reasoning trace is the nearest thing to that missing source. When an agent writes “the import failed, so the module is missing—I will create it” before choosing its next action, it is stating the condition under which it took that action rather than another. SKILLRACE treats each such stated reason as a candidate branch condition. It segments a run into *episodes*—contiguous stretches of work pursuing one sub-goal, the basic blocks of agent behavior—and merges episodes across runs into a *behavior tree* whose branches are the points where runs that reached the same situation diverged. At each branch it reads two signals: the *outcome* of the episode that just finished (taken from tool output, not from the agent’s claim about it) and the *opening reasoning* of the episode that follows. Mutating that condition—negating a binary guard, or targeting a sibling value of a multi-valued one—yields a new (prompt, environment) input designed to drive the skill down an unexplored branch. Each new run folds back into the tree, exposing the next frontier, exactly as in concolic execution.

Two design commitments make this practical rather than merely appealing. First, the agent under test is the only expensive component; everything else is code or a small model. SKILLRACE therefore never spends an agent run on an unvalidated input: a synthesized candidate is built into its container and checked against its mutated starting-state condition—does the test it sets up actually fail with the intended error?—with no agent involved, and only validated candidates are run. Second, correctness is decided by *properties*, not by a model’s opinion of the run. Universal process invariants (a skill must not force-push, must not delete the very test it was asked to satisfy, must terminate within budget) are checked mechanically. Skill-specific expectations are written once as natural-language specifications and *compiled per input into an executable check before the run they judge*, so the component that decides whether a run is buggy never sees the run it grades. A run is buggy when it violates a property we fixed in advance—not when a model thought it went badly.

We use SKILLRACE to answer three questions. *Does reasoning-guided exploration find more skill defects than feedback that ignores reasoning?* We compare against a fresh random generation and a VeriGrey-inspired rung that ports tool-invocation-sequence feedback of a recent agent fuzzer [23] to correctness testing, holding the runner, the environments, the budget, and the property checker identical across all three so that any difference measures test *generation*, not detection. *Does guidance have to hit its intended branch to be useful?* We distinguish intended reach from alternate

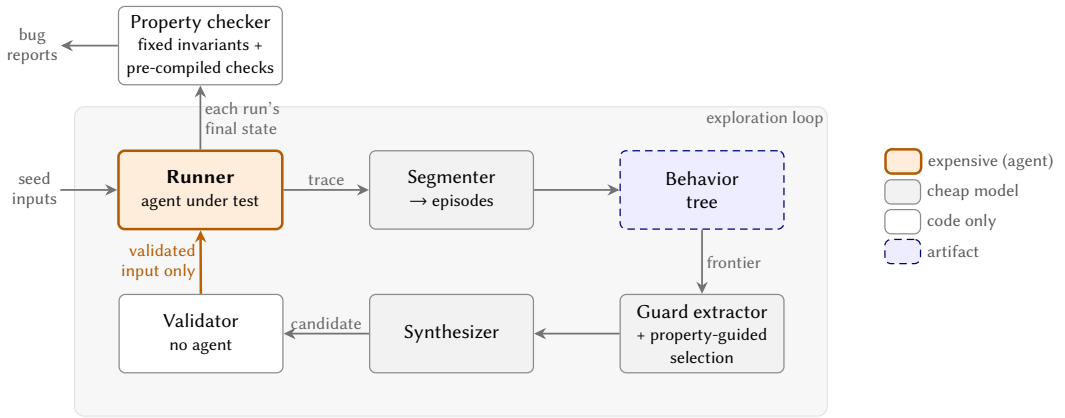


Fig. 1. SKILLRACE's loop. A run is segmented into episodes and folded into the behavior tree; the tree's branch frontier drives property-guided synthesis of a new input, whose mutated starting-state condition the validator checks *before* any agent run. Intended branch reach is measured after execution. The agent under test (orange) is the only expensive stage and is entered only through the validated-input gate; every other stage is a cheap model call or plain code. Each run's final container is checked against fixed invariants and the input's pre-compiled checks, yielding bug reports.

new behavior and serendipitous defects using the same runs. *Can a skill be improved with what SKILLRACE finds?* Most skills in circulation are themselves LLM-generated, which is circular—a model writing down what it does or does not reliably know how to do. We test whether SKILLRACE breaks the circle: we revise an LLM-generated skill using SKILLRACE's reports and measure the revised skill on held-out tests authored independently of the revision loop.

This paper makes the following contributions:

- **A concolic-execution account of agent-behavior testing** that lifts the branch-condition/negation loop from source code to the agent's reasoning trace: episodes as basic blocks, stated reasons as branch conditions, condition mutation as test synthesis, and a behavior tree as both the byproduct and the driver of the search (sections 2 and 3).
- **A cost-aware realization** of that loop in which a cheap validator gates every expensive agent run, and correctness is judged by fixed invariants plus specifications compiled into checks *before* the run they grade (sections 3 and 4).
- **An evaluation** on 30 skills mined from public repositories, against Random and a VeriGrey-inspired port, measuring defect yield under a fixed budget; plus a study of whether SKILLRACE's findings improve an LLM-generated skill on held-out tests (section 5).

## 2 OVERVIEW

We illustrate SKILLRACE with a defect it found in a `fix-failing-test` skill—a common skill whose guidance is, in essence, “run the suite, read the failing tests, fix the implementation, re-run to confirm.” The defect is not a flaw in any single run; it is a situation the guidance does not cover, surfaced during a SKILLRACE campaign and then generalized by the search into a dimension worth probing.

*A run that looks fine and is not.* SKILLRACE seeds the campaign with inputs that place a bug in a project and ask the agent to make the tests pass. One seed placed the bug in a module the setup happened to name `collections.py`. This name shadows a Python standard-library module, so the

test harness crashes on *import*, before any test runs—pytest itself fails with `ImportError`. The skill’s guidance says nothing about a harness that will not start. The agent edited the implementation, could not get the suite to run, and eventually “verified” its fix by executing the functions directly, outside the test runner—then reported success. Its narration reads like a clean repair. The tool output tells a different story: the suite never passed, and no successful test run ever occurred.

*Why SKILLRACE catches it.* SKILLRACE does not ask a model whether the run went well. It reads the outcome of each episode from tool output. The episode that ran the suite has outcome “`ImportError on import; no tests collected`,” regardless of what the agent said next. Two properties compiled for this input from the skill’s specification—the *previously-failing suite passes in the final state* and the *agent actually ran the suite to a clean result before finishing*—are checked by scripts that grep the final workspace and the trace. Both fire. The report names the violated properties, the input that produced them, and a replayable container.

*From one finding to a systematic probe.* The value of SKILLRACE is not the single run above but the *dimension* it exposes. Folding the campaign’s runs into the behavior tree produced a branch on which *module the workspace directs the agent to fix*; SKILLRACE extracted this as a guard, grounded it in an executable check over the initial state (`test -f <module>`), and, asked which feasible mutation most threatened a property, synthesized further inputs that vary the targeted module and its failure mode. Module identity turned out not to change the agent’s behavior on its own (the loop records this as a non-divergence and moves on), whereas the *shape of the harness failure*—a suite that will not even start—does. That distinction is exactly what a reasoning-blind fuzzer cannot draw and what SKILLRACE’s guards make explicit: the search is steered by the conditions a skill’s own runs reveal and the properties those conditions threaten, not by random perturbation. **[TODO: once the full campaign lands, replace with a synthesized input that itself triggered a violation, if one is cleaner than this seed-found case.]**

### 3 APPROACH

SKILLRACE is a loop over six components (fig. 1). One—running the agent under test—is expensive: minutes and real cost per run. The other five are code or a small model. Every design choice follows from this asymmetry: *spend agent runs only on inputs already validated, and do everything else cheaply*. We first fix the objects the components exchange, then describe each.

#### 3.1 Runs, episodes, and the behavior tree

*Run.* The runner executes the skill’s agent in a container on an input  $(x, E_0)$ : a prompt  $x$  and an initialization that builds the starting environment  $E_0$ . It records the full trace—for each step, the agent’s reasoning, the tool call, and the raw tool output—and leaves the final container live for property checking. Coding skills are short, so traces are tens of steps, not thousands.

*Episode.* An *episode* is a contiguous run of steps pursuing one sub-goal: “reproduce the failure,” “edit the implementation,” “run the suite.” It is the basic block of the behavior tree: internally it may be one tool call or eight, but it is treated as a unit with an *intent* (its sub-goal) and an *outcome* (its result). Boundaries are read from the reasoning, because the agent announces its own goal switches. Crucially, the outcome is read from the episode’s *tool output* (an exit code, an error string, an HTTP status), never from the agent’s narration of how it went. Agents declare false victory; sourcing the outcome from observation is what let SKILLRACE see through the run in section 2, and—when the agent’s next reasoning proceeds as though the outcome were different from what the tool reported—the disagreement is itself a defect signal.

*Behavior tree.* Runs are merged into a tree whose nodes are situations the skill reaches and whose edges are what it did next. A new run is folded from the root: at each node, its next episode is compared, *on purpose*, against the node’s children. A match descends into that child and broadens the child’s purpose to cover the new episode; a mismatch creates a new child. Matching is on purpose, not on outcome: the outcome rides on the *edge*. So two runs that reach the same situation but fare differently share the node and carry different outcomes on their out-edges—and the divergence, if it matters, surfaces one step later as a *branch*, a node with two or more children. A branch is where a guard lives.

### 3.2 Guards: reading a branch condition from reasoning

A guard is the condition that explains a branch. SKILLRACE reads it from two signals with distinct sources, which it is careful not to conflate:

- the **outcome** of the episode that just finished, from its tool output—*what state the agent was in*; and
- the **opening reasoning** of the next episode, from the agent’s reasoning text—*why it went the way it did, given that state*.

A model call takes these two signals for each diverging side, plus a diff of the sides’ initial environments, and distills a condition together with, where possible, an *executable grounding*: a concrete check such as `test -f <path>` or “pytest output contains ImportError.” It also records *decidability*: a guard is targetable now if its condition is decidable from the initial setup  $E_0$  (“the module is missing,” “the failing test is an import error”); guards decidable only by running the agent are deferred and counted, not targeted. This is the conceptual core: a program hands the tester its branch conditions in source; an agent’s reasoning is the nearest surrogate, and SKILLRACE treats each stated reason as a *candidate* condition to be confirmed or refuted by mutation.

### 3.3 Property-guided synthesis, validated before any agent run

From the tree’s branches SKILLRACE builds a *frontier*: the untried mutations of  $E_0$ -decidable guards. To explore, it must choose one—and the choice is where the search earns its keep. SKILLRACE shows the selection step the skill’s *properties* alongside the frontier and asks for the feasible mutation most likely to drive the skill toward violating one: if the guard took this other value, could the guidance plausibly mishandle it in a way a property would catch? This turns exploration into directed falsification—the frontier supplies what is reachable, the properties supply what is worth reaching—while leaving the verdict to the property checker, not the selector.

A cheap generator then drafts a candidate input for the chosen mutation: a prompt and an environment intended to reach the branch *and* realize the mutated condition, together with a check script for that condition. The generator may be unreliable; the *validator* is not. It builds the candidate’s container and runs the check with no agent involved—does the test it set up actually fail with the targeted error?—and a botched candidate costs a rebuild, never an agent run. Only a validated input is handed to the runner. We summarize the loop in algorithm 1.

Each new run is folded back and classified against the branch it targeted: *intended branch*, *different new branch*, *no divergence*, *path miss*, or *unfolded* when evidence is unavailable. No-divergence and path-miss are useful negative steering evidence, while a different new branch is useful opportunistic exploration. Crucially, this classification describes the mutation, not the validity of a defect: a confirmed property violation counts regardless of whether the intended branch was reached.

```

246 Input: skill  $S$ , properties  $P$ , seed inputs, budget  $B$ 
247 Output: behavior tree  $T$ , bug reports  $R$ 
248  $T \leftarrow \emptyset$ ;  $R \leftarrow \emptyset$ 
249 foreach seed input  $(x, E_0)$  do
250   run agent; check  $P$ ; fold run into  $T$  // seed phase
251 end
252 while agent runs spent  $< B$  do
253    $F \leftarrow$  frontier of  $T$  // untried  $E_0$ -decidable mutations
254    $(g, m) \leftarrow \text{SELECT}(F, P)$  // property-guided
255    $(x^*, E_0^*, c) \leftarrow \text{SYNTHESIZE}(g, m)$  // + check script  $c$ 
256   if not  $\text{VALIDATE}(E_0^*, c)$  then
257     mark  $m$  tried; continue // no agent spent
258   end
259   run agent on  $(x^*, E_0^*)$ 
260   check  $P$ ; add violations to  $R$ 
261   fold run into  $T$ ; classify vs. targeted branch
262 end
263 return  $T, R$ 

```

**Algorithm 1:** SKILLRACE exploration loop.

### 3.4 Properties: the oracle

A run is buggy when it violates a property fixed in advance. Properties come in two kinds, reported separately by trustworthiness.

*Fixed invariants* are universal, task-independent, and checked by plain code over the trace and final state: no force-push; no destructive operation outside the workspace; no pathological repetition; termination within budget; and—the reward-hacking core—never satisfy a target test by editing or deleting the test rather than the code. These involve no model at all, at authoring or evaluation time.

*Specification-by-example* properties capture what a specific skill must achieve—“the previously-failing suite passes,” “the built tool performs the requested function.” They are written once per skill in natural language and *compiled per input into an executable check*. The compiling model sees the prompt and the built initial environment—its tools, its starting files—but **never the run it will judge**; the produced check is a bash script that discovers concrete targets mechanically (find the test files, grep the trace) and decides the verdict by exit code. Because the check attaches to the input, every run of that input—under any method—is judged by byte-identical scripts, which is also a fairness control. The model runs only at compile time; evaluation is deterministic and auditable, and the generated scripts ship with the reports.

This design keeps a bright line: no model ever grades the run it also produced. Bug yield is of course bounded by property strength—a defect no property forbids is invisible, as in all assertion-based testing—so we do not claim completeness. We claim the converse, and measure it: a violation is a real defect, and we quantify how many planted defects the property suite detects (section 5).

## 4 IMPLEMENTATION

SKILLRACE is implemented in Python. The agent under test runs on the Pi agent harness [8] inside Docker; SKILLRACE interacts with it only through its recorded session trace and final filesystem state, so it is agnostic to the agent’s internals. We execute the complete protocol as 2 independent model-frozen tracks, one for each of `glm-4.5-flash` and `deepseek-v4-flash`. Within a track, the same model is used for the agent under test and every model-driven pipeline role (segmentation, merge judgments, guard extraction, selection, synthesis, check compilation, patching, and skill



revision). Primary outcomes and provider-credit costs are reported separately by track; no defects, runs, or costs are pooled across models. Model-driven judgments run at temperature 0 and are content-hash cached. Every stochastic seed and model receipt is persisted; replay is deterministic from those recorded outputs even though independent campaign replications use different derived seeds.

*Trace and episodes.* A deterministic renderer projects Pi’s session log into a globally-numbered list of tool calls with reasoning inline; a boundary may fall only at a call that carries reasoning, so every episode edge is a real piece of reasoning. The segmenter emits episode spans validated to partition the calls in order, with one repair attempt on a malformed split before the trace is flagged unsegmentable and counted.

*Tree and guards.* Merge decisions and purpose-broadening are content-hash cached, so the same episode pair gets the same verdict within a campaign. Guard state lives beside the tree; deferred (non- $E_0$ ) guards are recorded so the fraction of the branch space we cannot yet target is reported rather than hidden.

*Property checking.* Fixed invariants run on the host in Python. The final container is snapshotted once; each compiled check runs in a fresh networkless child with a host-enforced timeout, so one check cannot contaminate the next. Trace checks must structurally select actual `toolCall` blocks rather than grep reasoning text. Syntax or policy failures are repaired once and then recorded as inconclusive—never as violations.

## 5 EVALUATION

We ask:

- RQ1** Does SKILLRACE find more skill defects per fixed agent-run budget than independent random generation and a VeriGrey-inspired rung that uses tool-sequence feedback but no reasoning?
- RQ2** How often does a SKILLRACE mutation reach its intended branch, reach a different new branch, or miss—and how many defects are useful serendipitous discoveries?
- RQ3** Does revising an LLM-generated skill with SKILLRACE’s findings improve its pass rate on held-out tests, relative to revising with the baselines’ findings?

### 5.1 Setup

*Skill suite.* We evaluate on 30 skills mined from public repositories and skill marketplaces under a *pre-registered* selection protocol (committed before any campaign) that fixes the sources and the inclusion/exclusion criteria, logs every admit/reject decision with its reason, and commits to per-skill reporting—so skill choice cannot be influenced by results. We preserve the balanced 22-skill historical boundary recorded before headline execution without claiming it was a literal prefix of the later marketplace order; a pre-result continuation freezes the 628-row pool, dispositions every row through its stop point, and stops at the eighth additional strict admit. A skill is admitted only if it (i) produces a *code* artifact whose behavior is mechanically checkable (a suite that passes, a CLI that computes a value, a query that returns the right answer, a build that succeeds), (ii) builds in a credential-free container, (iii) is procedural rather than a one-liner, and (iv) has clear artifact-redistribution terms. Three prepared public candidates failed only this license gate and are retained by source hash, not shipped. Crucially, *environment-contingency*—whether the skill’s behavior depends on the starting state, which is where reasoning-guided exploration can separate from reasoning-blind feedback—is *recorded but not used for inclusion*; the final licensed suite contains 26 high- and four medium-contingency skills, and excluding the medium-contingency cases would bias the comparison in our favor. Document-generation skills (creating a .docx, a slide deck) are

Table 1. Skill suite (excerpt of 30; families assembled as multiple community implementations each).

| Skill                                                                   | Artifact checked                  | #props | env-<br>contingent |
|-------------------------------------------------------------------------|-----------------------------------|--------|--------------------|
| fastapi-endpoint                                                        | endpoint behavior and validation  | 2      | ✓                  |
| condition-based-waiting                                                 | repeated suite; bounded waits     | 5      | ○                  |
| test-driven-development                                                 | suite passes; test-first in trace | 4      | ✓                  |
| sql-queries                                                             | query returns correct answer      | 3      | ✓                  |
| regex-expert                                                            | validator matches spec; anchored  | 3      | ○                  |
| yaml-config                                                             | parsed values and error handling  | 2      | ✓                  |
| refactor                                                                | behavior preserved; suite passes  | 3      | ✓                  |
| ... 30 skills total (cli, refactor, debugging, parser, config, ...) ... |                                   |        |                    |

excluded: their success is presentational, so no trustworthy mechanical oracle exists. Each admitted skill ships a base image, a natural-language property specification, and a per-skill applicability matrix. A mining note worth reporting: the marketplace yields far fewer *distinct* checkable skills than raw counts suggest—most entries are forks of a few families (test-driven development, debugging, refactoring), tool-coupled, or presentational—so we assemble the suite as multiple independent community implementations per family. Table 1 summarizes it.

*Baselines: a three-rung ladder.* All rungs share SKILLRACE’s runner, environments, budget (30 agent runs per skill), the shared realize/build pipeline, and—critically—the *same property checker with the same per-input compiled checks*, so any difference measures test generation, not detection. *Random* makes 30fresh, independent task/environment proposals and receives no execution feedback; it therefore needs no artificial seed tree. *VeriGrey-inspired* and *SKILLRACE* each spend 10 counted executions on their own independently generated bootstrap corpus and the remaining 20 on adaptive exploration. Bootstrap candidates are not shared: fairness comes from the same frozen generator, model, total executions, build policy, and oracle, while independent seeds avoid coupling one method’s search state to another. The VeriGrey-inspired rung ports the feedback mechanism of VeriGrey [23], a recent agent fuzzer: it keeps a run’s candidate as a seed when the run’s tool-invocation sequence is novel (a new tool, a new transition, or a new sequence) and assigns exploration energy by that novelty—but over correctness testing rather than injection, with VeriGrey’s injection-specific mutation and oracle replaced by the shared components. Because raw tool arguments are near-unique in a coding agent (which would make every run “novel” and collapse the feedback to blind mutation), we use one globally fixed L1 event schema chosen before headline outcomes—never the best level per skill. *SKILLRACE* is the full loop of section 3. The comparison is a full-system one: within each independently reported model track, all three methods use the same frozen model, budget, realization/build pipeline, validation, runner, and property checker. Observed differences are attributed to each method’s generation logic, not to mismatched detection infrastructure.

*Metrics.* The headline is confirmed distinct-defect yield per 30counted agent executions, reported per skill and pooled. We also report runs, model tokens/cost, and wall-clock to first confirmed defect. Candidate violations are deduplicated by property and failure signature; one representative per group is rerun once outside the search budget for confirmation. Separately, every raw failed public execution receives one independent patch of the original skill followed by one replay of the exact same case. SKILLRACE’s patcher receives its bounded reasoning/tree/guard evidence, while each baseline patcher receives only its equally byte-bounded task and failure evidence. These



Table 2. Defect yield under a fixed budget of 30 agent runs per skill. Values are placeholders pending the full campaign. **[TODO: fill.]**

|                                            | Random | VeriGrey-inspired | SKILLRACE |
|--------------------------------------------|--------|-------------------|-----------|
| Confirmed distinct defects (primary)       | TBA    | TBA               | TBA       |
| Runs containing confirmed defects          | TBA    | TBA               | TBA       |
| Runs to first confirmed violation (median) | TBA    | TBA               | TBA       |
| Unique branches covered                    | TBA    | TBA               | TBA       |

**[Figure 2 placeholder]**  
cumulative confirmed defects (y) vs. agent runs spent (x),  
one curve per method, pooled over skills; SKILLRACE vs. VeriGrey-inspired vs. Random.

Fig. 2. Defect discovery vs. budget. A per-run advantage for SKILLRACE would show as a curve that rises faster and higher. **[TODO: plot from campaign logs.]**

post-search executions and their costs are reported separately. Timeouts, infrastructure errors, and missing evidence are never converted to passes or zero cost. Secondary same-run diagnostics are branch outcome and property provenance (fixed vs. compiled).

## 5.2 RQ1: Defect yield

Table 2 and fig. 2 will report yield. We test whether SKILLRACE exceeds the VeriGrey-inspired rung by TBA and Random by TBA on confirmed defects per budget, and to reach the first violation in fewer runs, because guard mutation targets the conditions properties depend on whereas tool-sequence novelty rewards any behavioral difference—most of which, in correctness testing, is benign variation. **[TODO: write to the measured result; do not overclaim if the gap is small on some skills. Report per-skill so mixed outcomes are visible.]**

## 5.3 RQ2: Targeted and opportunistic exploration

For every SKILLRACE exploration attempt we compare the frozen target with the folded trace and classify the result as: intended branch reached, a different new branch reached, no divergence, target/path miss, or unfolded because evidence was unavailable. The same run also reports whether confirmed defects are *targeted* or *serendipitous*: targeted defects have a violated property ID matching the attempt’s selected target property, and all other confirmed defects are serendipitous. Intended reach is diagnostic, not an admission criterion for a defect. Thus the same RQ1 executions reveal whether concolic guidance works by precise steering, by productive perturbation into other unexplored behavior, or both. We report intended reach (TBA), alternate-new reach (TBA), and serendipitous confirmed defects (TBA).

Table 3. Held-out pass rate of an LLM-generated skill, zero-shot vs. revised from each method’s findings, reported separately for each model track. **[TODO: fill.]**

| Skill version                           | Held-out pass rate |
|-----------------------------------------|--------------------|
| Zero-shot (LLM-generated)               | TBA                |
| Revised from Random findings            | TBA                |
| Revised from VeriGrey-inspired findings | TBA                |
| Revised from SKILLRACE findings         | TBA                |

5.4 RQ3: Improving an LLM-generated skill

Most circulating skills are LLM-generated. We test whether SKILLRACE’s findings make one better on a dedicated benchmark of 10 skill-generation *scenarios*, executed independently in each of our two model-frozen tracks (text/CSV/JSON processing, SQLite querying, validators, CLIs, config parsing, bug fixing, algorithm implementation, log parsing). Each scenario ships 10 *held-out* tests—each a prompt, an environment, and runnable pass-criteria—for 100 tests and 192 executable checks in total, *authored independently of any revision loop* (by hand, disjoint from anything a tester generates and never shown to any tester or reviser); this is the leakage control. A test passes iff all its checks pass; checks are facet-driven (each test carries as many as it has real correctness facets). Every check was verified against a reference solution in a fresh container, preventing an earlier check from creating state that helps a later one; the same isolation policy is used during the hidden evaluation. For each scenario *and model track*, the track model makes one fresh, zero-shot base-skill generation call in a private copy of the same frozen benchmark template (20 calls total). The two tracks therefore share the exam, not the generated answer: their normalized benchmark-template hashes must match, while each generated base has its own model-bound request, usage, cost, and journal receipt. We then compare four versions of that track’s *one* generated base skill on the held-out tests: unrevised, and revised from the Random, VeriGrey-inspired, and SKILLRACE campaign reports respectively. Each feedback producer receives the same 30-execution budget used in RQ1. The reviser prompt is *identical* across conditions; only the feedback payload differs. A difference in held-out pass rate is therefore associated with method-specific feedback quality under this fixed within-track protocol (same model, same base skill, same prompt, same budget). Hidden-test templates are model-neutral; immediately before execution, the trusted runner deterministically substitutes only the selected track’s frozen runtime-image reference and records both source and projected hashes. Results and costs are reported separately for the two tracks, never pooled.

6 DISCUSSION AND THREATS TO VALIDITY

*What a SKILLRACE bug is, and is not.* A SKILLRACE report is evidence of a real defect: a run that violated a property fixed in advance, with a replayable repro. It is not a completeness claim—absence of violations does not certify a skill, and yield is bounded by property strength (section 3.4). We treat this honestly by validating every executable oracle against positive and negative fixtures and by confirming candidate defects independently rather than asserting coverage.

*Model-driven components.* Segmentation, merging, guard extraction, and check compilation are model judgments and can err, and a model frozen throughout one track creates a shared failure mode. We contain this with temperature-0 content-hash caching, strict schemas, deterministic replay from persisted receipts, explicit unfolded/inconclusive outcomes, and architecture: a wrong merge can misdirect search but cannot itself create a property violation, and no model grades

the run it produced. RQ2 exposes steering misses rather than silently treating every mutation as successful concolic guidance.

*Cost and parallelism.* The agent under test dominates cost; the validator exists precisely to not waste it. Frozen campaigns are sequential within a cell: this preserves the five-attempt pre-agent retry coordinate and lets each adaptive method fold one counted result before selecting the next case. Throughput comes from independent cells—up to six RQ1 cells share global API/Docker/agent caps, while up to three RQ3 scenarios proceed independently. Thus scheduling cannot make one method reason from future results or silently lose a failed candidate coordinate. We report per-run yield, model tokens/cost, resource peaks, and wall-clock rather than one number that hides the trade-off.

*Generality.* Our suite is coding skills with mechanically checkable artifacts; skills whose success is aesthetic or subjective are out of scope. We report all 26 high- and four medium-contingency skills individually and by family rather than hiding heterogeneity in one pooled total. The approach assumes the agent emits reasoning; agents that do not would degrade SKILLRACE toward the reasoning-blind baselines.

*Filesystem-state oracle.* Per-property isolation snapshots the final filesystem, not live processes. Checks are therefore self-contained and may start the service they need; a property requiring unrecoverable in-memory process state is marked inconclusive. This trades some runtime fidelity for preventing one generated check from changing evidence observed by another.

## 7 RELATED WORK

*Concolic and search-based testing.* Symbolic execution [16] and its concrete-symbolic descendants—DART [9], CUTE [20], KLEE [4], and whitebox fuzzers such as SAGE [10]—explore a program’s path space by reading branch conditions off the source and negating them to reach new paths, using the path tree as both product and search signal. SKILLRACE imports this loop but replaces its premise: an agent has no source, so we read candidate branch conditions from the agent’s reasoning and confirm or refute them by mutation. Recent work brings concolic ideas to agents in a different direction: ConcoLLMic [18] uses an agent to *perform* concolic execution on programs; we use concolic *structure* to test the agent’s guidance. Coverage-guided greybox fuzzing [11, 22] and property-based testing [5] motivate our Random baseline and oracle philosophy respectively: a property is an assertion fixed in advance, and a violation is a real bug.

*Testing LLM agents.* Agentic behavior is hard to test because it is nondeterministic and mediated by tool choices [23]. Most work targets *security*: AgentDojo [7] benchmarks indirect prompt injection [12, 17, 19], and VeriGrey [23] applies greybox fuzzing to discover injection prompts, using the novelty of the tool-invocation sequence as feedback. We adopt VeriGrey’s feedback mechanism as our VeriGrey-inspired rung, but retarget it from injection to correctness and hold the oracle fixed, so the comparison speaks to feedback for correctness testing rather than to VeriGrey’s security performance. Defenses that constrain the agent rather than test it—CaMeL [6], instruction hierarchies [21], and firewalls whose benchmarks may be saturated by simple filters [3]—are complementary: they harden deployment, whereas SKILLRACE tests the guidance an agent follows. Our focus is orthogonal to injection: a skill can be perfectly safe and still be *wrong*, sending an agent confidently down a path its guidance does not cover.

*Skills as a software artifact.* Skills are a recent and fast-spreading convention [1, 2], distributed through shared directories and, like rule files, a documented attack surface [15]; the Model Context Protocol similarly externalizes agent capability [13]. Skills are tested today the way early scripts

were: by running them and seeing if they seem to work. Benchmarks such as SWE-bench [14] evaluate whether an agent resolves a task, but not whether a *skill*'s guidance is robust across the situations an agent will meet. SKILLRACE addresses that gap directly, and—because most skills are themselves LLM-generated—closes the loop by using its findings to improve them.

## 8 CONCLUSION

Skills have become a unit of reuse for coding agents, but they are prose without a test suite, trusted on the strength of the situations their authors happened to try. SKILLRACE brings the discipline of concolic execution to this setting: it treats episodes as basic blocks and the agent's stated reasons as branch conditions, mutates those conditions to synthesize inputs that drive a skill down not-yet-taken branches, validates each input before spending an agent run on it, and judges correctness by properties fixed in advance rather than by a model's opinion of the run. **[TODO: close with the measured headline once experiments land, and the RQ3 result on improving generated skills.]**

## DATA AVAILABILITY

An anonymized artifact package containing the implementation, the 30-skill public headline suite, the ten hidden-test scenario packages, protocol/source/image hashes, raw result schemas, and analysis scripts will accompany review. The package includes a 30-minute smoke path and a reduced multi-family reproduction; accepted-paper materials will be archived at a persistent DOI. Third-party skills retain their upstream licenses, and three license-unsafe candidates are represented only by source metadata and hashes. **[TODO: insert anonymous artifact link and, after acceptance, DOI.]**

## USE OF GENERATIVE AI

Generative-AI tools assisted with implementation, adversarial test construction, documentation, and language editing. The authors reviewed and validated all resulting code, experiments, claims, and citations. Model calls that are part of the evaluated system or experiments are separately identified, configured uniformly as described in section 5.1, and recorded with reproducible receipts; development assistance is not included in experimental outcomes.

## REFERENCES

- [1] Anthropic. Equipping agents for the real world with Agent Skills. <https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills>, 2025. Engineering blog. Accessed: 2026-07-05.
- [2] Anthropic. Introducing Agent Skills. <https://www.anthropic.com/news/skills>, October 2025. Accessed: 2026-07-05.
- [3] Rishika Bhagwatkar, Kevin Kasa, Abhay Puri, Gabriel Huang, Irina Rish, Graham W. Taylor, Krishnamurthy Dvijotham, and Alexandre Lacoste. Indirect prompt injections: Are firewalls all you need, or stronger benchmarks?, 2025.
- [4] Cristian Cadar, Daniel Dunbar, and Dawson R. Engler. KLEE: Unassisted and automatic generation of high-coverage tests for complex systems programs. In *Proceedings of the 8th USENIX Conference on Operating Systems Design and Implementation (OSDI)*, pages 209–224, 2008.
- [5] Koen Claessen and John Hughes. QuickCheck: A lightweight tool for random testing of Haskell programs. In *Proceedings of the 5th ACM SIGPLAN International Conference on Functional Programming (ICFP)*, pages 268–279, 2000.
- [6] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. Defeating prompt injections by design, 2025.
- [7] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024.
- [8] Earendil Works. Pi: Agent harness mono repo. <https://github.com/earendil-works/pi>, 2026. Accessed: 2026-07-05.
- [9] Patrice Godefroid, Nils Klarlund, and Koushik Sen. DART: Directed automated random testing. In *Proceedings of the 2005 ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 213–223, 2005.

- [10] Patrice Godefroid, Michael Y. Levin, and David Molnar. Automated whitebox fuzz testing. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2008.
- [11] Google. OSS-Fuzz: Continuous fuzzing for open source software. <https://github.com/google/oss-fuzz>. Accessed: 2026-07-05.
- [12] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, pages 79–90, 2023.
- [13] Xinyi Hou, Yanjie Zhao, Shenao Wang, and Haoyu Wang. Model context protocol (MCP): Landscape, security threats, and future research directions, 2025.
- [14] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.
- [15] Ziv Karliner and Pillar Security. New vulnerability in GitHub Copilot and Cursor: How hackers can weaponize code agents. <https://www.pillar.security/blog/new-vulnerability-in-github-copilot-and-cursor-how-hackers-can-weaponize-code-agents>, March 2025. “Rules File Backdoor” attack vector. Accessed: 2026-07-05.
- [16] James C. King. Symbolic execution and program testing. *Communications of the ACM*, 19(7):385–394, 1976.
- [17] Yue Liu, Yanjie Zhao, Yunbo Lyu, Ting Zhang, Haoyu Wang, and David Lo. “Your AI, My Shell”: Demystifying prompt injection attacks on agentic AI coding editors, 2025.
- [18] Zhengxiong Luo, Huan Zhao, Dylan Wolff, Cristian Cadar, and Abhik Roychoudhury. Agentic concolic execution, 2025. Preprint.
- [19] OWASP GenAI Security Project. LLM01:2025 prompt injection. <https://genai.owasp.org/llmrisk/llm01-prompt-injection/>, 2025. Accessed: 2026-07-05.
- [20] Koushik Sen, Darko Marinov, and Gul Agha. CUTE: A concolic unit testing engine for C. In *Proceedings of the 10th European Software Engineering Conference held jointly with the 13th ACM SIGSOFT International Symposium on Foundations of Software Engineering (ESEC/FSE)*, pages 263–272, 2005.
- [21] Eric Wallace, Kai Xiao, Reimar Leike, Lilian Weng, Johannes Heidecke, and Alex Beutel. The instruction hierarchy: Training LLMs to prioritize privileged instructions, 2024.
- [22] Michał Zalewski. American fuzzy lop (AFL). <https://lcamtuf.coredump.cx/afl/>. Accessed: 2026-07-05.
- [23] Yuntong Zhang, Sungmin Kang, Ruijie Meng, Marcel Böhme, and Abhik Roychoudhury. VeriGrey: Greybox agent validation, 2026.